

INTERNSHIP PROJECT REPORT

Project Title

RFQ2BOQ — Automated Bill of Quantities Extraction from Construction Tender Documents using NLP

Choda Srujan Sai

Roll No. 23110081

IIT Gandhinagar

Internship at SWA Consultancy Pvt. Ltd.

Under the guidance of Dr. Sudish Mishra

Certificate

This is to certify that **Choda Srujan Sai** (Roll No. 23110081) has successfully completed the internship project titled "**RFQ2BOQ — Automated Bill of Quantities Extraction from Construction Tender Documents using NLP**" under the guidance of **Dr. Sudish Mishra** during the internship period at **SWA Consultancy Pvt. Ltd.** The work presented in this report is original and has been completed as part of the internship requirements.

Guide Signature: Dr. Sudish Mishra

Date: 15-07-2026

Organization Representative: SWA Consultancy Pvt. Ltd.

Declaration

I hereby declare that this project report entitled "**RFQ2BOQ — Automated Bill of Quantities Extraction from Construction Tender Documents using NLP**" is my original work carried out during my internship. The information and data presented in this report are authentic to the best of my knowledge and belief.

Student Name: Choda Srujan Sai

Enrollment Number: 23110081

Signature:



Date: 15-07-2026

Reviewed by (Buddy): Dr. Sudish Mishra

Buddy Signature: Sudish mishra

Date: 16-07-2026

Acknowledgement

I express my sincere gratitude to SWA Consultancy Pvt. Ltd. for providing me with the opportunity to work on this project. I thank my project mentor and the HR team for their continuous guidance, encouragement, and support throughout the internship. Their feedback during the review meeting on 11th June 2026 in particular sharpened the direction of the project and its data-quality requirements. I also thank IIT Gandhinagar for facilitating this internship as part of my academic program.

Table of Contents

1. Abstract
2. Introduction
3. Organization Profile
4. Problem Statement
5. Objectives
6. Scope of the Project
7. Literature Review / Background
8. Methodology
9. Tools and Technologies Used
10. System Requirements
11. Project Planning
12. Implementation
13. Testing and Validation
14. Results and Discussion
15. Challenges Faced
16. Learning Outcomes
17. Future Scope
18. Conclusion
19. References
20. Appendices

Abstract

Construction tenders issued in India (Requests for Quotation, or RFQs) typically arrive as unstructured PDF or Excel files running anywhere from a few pages to over a hundred, mixing eligibility criteria, technical specifications, and Bill of Quantities (BOQ) tables together with no consistent layout. Right now, estimators at SWA Consultancy read through these documents by hand and transcribe every line item — material, quantity, unit, grade, standard — into a usable BOQ. It works, but it is slow and a single missed row can have real commercial consequences.

This project builds a pipeline that automates that conversion, turning a raw tender document into a structured, unpriced BOQ in Excel and JSON. The engineering priority throughout was fidelity: the client's requirement was that no row should ever silently disappear during conversion, and anything the system isn't confident about should be flagged for a human to check rather than guessed at.

The way this was built is worth describing honestly, because it shaped everything else. I worked as an orchestrator directing a set of AI coding agents rather than writing every line of code myself — breaking the project into precisely specified tasks, dispatching them, and then independently checking the results before accepting anything as done. That last part turned out to matter a great deal. More than once, an agent reported a task as "100% complete" when what had actually happened was that a test's expected output had been quietly edited to match broken code, or a measurement script had a bug that made things look better than they were. Catching this required treating every "done" claim as a hypothesis to verify, not a fact to accept.

As of this report (numbers re-checked directly against the repository on 2026-07-16 with a full `audit_fidelity_per_doc.py --all` run), the pipeline runs end to end: it validates extracted materials against the client's own GeM product catalog (19 products, ingested with full provenance), routes large documents structure-first rather than scanning every page blindly, and includes a fidelity-audit tool built specifically so completeness can be checked per document instead of just claimed. Under that independent auditor, the ten-document sacred reference set is **10/10 PASS** and the broader 33-document BOQ-bearing corpus is **33/33 PASS** (capture / missing / extra all zero-miss on each entry in `results/fidelity/summary.json`). That is the R1 *row-capture* bar for the current corpus — it is **not** the same claim as domain NER F1 or “never wrong on any future document.” A separate completeness harness can also report zero silently dropped rows when low-confidence extractions are counted as flagged rather than matched; content-match on a smaller independent sample of four Excel enquiries was **82.5%** (66 of 80 rows) and must not be conflated with the auditor PASS rate. On unstructured free text, real NER F1 remains ~0.43 until owner-verified BIOES gold exists.

A 127-document real tender corpus has been collected and split into a frozen train/test set. The largest remaining piece of work is training an actual domain-specific named-entity-recognition model, which depends on human-reviewed annotation data that does not yet exist in verified form — the tooling to produce it is built, but the review itself hasn't happened. This report tries to give an honest account of what was actually delivered, the discipline that was needed to keep the AI-agent-driven process trustworthy, and what genuinely remains.

1. Introduction

Government bodies (state power utilities, ISRO, and similar organizations) and private firms in India both issue construction tenders that routinely run fifty to over a hundred pages, mixing eligibility criteria, technical specifications, and one or more BOQ tables into a single document with no fixed structure. An estimator has to find the relevant section, read every row, and transcribe descriptions, quantities, units, grades, and standards by hand. It works, but it's slow, and a missed or mis-transcribed row translates directly into a costing error.

RFQ2BOQ automates that conversion. A PDF or Excel tender goes in; a structured, unpriced BOQ comes out in Excel and JSON, with every row traceable back to where it came from in the source document, and anything uncertain flagged instead of silently guessed.

The client for this project, SWA Consultancy, processes real construction and HVAC/insulation tenders as part of its regular business, and had already submitted its own product catalog to the Government e-Marketplace (GeM) portal — a standardized reference this project uses directly for validation. The expected payoff is straightforward: less manual transcription time, a fidelity guarantee that actually holds up in a business context with zero tolerance for lost data, and a system that keeps improving as more real tender data gets reviewed.

2. Organization Profile

SWA Consultancy Pvt. Ltd. is based at 303 Safal Prelude, Prahladnagar, Ahmedabad, and provides consultancy services in the construction, HVAC, and insulation domain, including tender response and estimation support. This internship sat within the company's technology/automation effort, building internal tooling to speed up tender-response workflows.

SWA works directly with real construction and industrial tender documents from clients that include government power utilities, aerospace and defense organizations, and private industrial firms. The manual effort of turning these tenders into structured Bills of Quantities was the specific problem this internship was asked to address.

3. Problem Statement

BOQ extraction from tender PDFs and Excel files is currently a manual process: an estimator reads through the whole document, finds the BOQ table or tables (which in large tenders can be split across multiple sections or annexures), and transcribes each line item by hand.

This problem persists because tender documents aren't standardized. No two tenders lay out their BOQ tables the same way, and in large tenders the actual BOQ content is often buried among dozens of pages of eligibility criteria and boilerplate compliance text. The result is that manual extraction scales linearly with tender size — a bigger tender just means more hours — and every transcription error, whether a missed row or a misread quantity, carries direct commercial risk.

What's needed is an automated pipeline that matches the completeness of manual transcription and can prove that it does, fast enough to process a hundred-page tender in under a minute, with a built-in way to flag anything it isn't confident about rather than quietly getting it wrong.

4. Objectives

The primary objective was to build a pipeline that converts real construction tender RFQs into a structured, unpriced Bill of Quantities, with a fidelity guarantee that uncertain rows get flagged rather than silently dropped.

Alongside that, five secondary objectives shaped the work:

1. Validate extracted materials against the client's own GeM portal product catalog as an authoritative reference.
 2. Implement structure-first extraction for large tenders — find the relevant BOQ section by scanning the document outline before running the more expensive extraction step, the way a human estimator would.
 3. Build the underlying data foundation: a real, diverse corpus of tender documents, plus the tooling to turn it into human-verified training data for a domain-specific NER model.
 4. Establish a verification discipline strong enough to catch fabricated or self-reported "complete" claims, whether from automated tooling or from the AI coding agents used during development.
 5. Deliver a usable Streamlit interface so an uploaded tender and its extraction results, including any flagged rows, are inspectable by someone without a technical background.
-

5. Scope of the Project

The pipeline covers PDF and Excel ingestion, structure-aware section routing, rule- and dictionary-based entity extraction across eight fields (material, quantity, unit, grade, standard, dimension, location, action), GeM catalog validation, BOQ row assembly with unit normalization, Excel/JSON export with flagged rows visible, a fidelity-audit tool, and an annotation-review tool for turning draft training candidates into verified labeled data.

The intended users are SWA's internal estimators, who need a starting-point BOQ from a tender document rather than a finished, priced one. The output is deliberately unpriced — rate, amount, and costing fields are out of scope by the client's own decision — and the project stops at the single extraction tool rather than growing into a full SaaS platform, multi-tenant system, CAD/drawing analysis tool, or anything with voice input; that boundary was set deliberately, at the client's direction.

Two things were assumed going in and held up in practice: that the client would supply the GeM catalog export and additional real tender documents, and that the client would provide human review time for annotation verification.

Honestly, as of this report, there are real limitations. The independent fidelity auditor now passes **10/10** on the sacred reference set and **33/33** on the BOQ-bearing corpus (row capture under the locked source-truth ruler), but that does not mean content-match F1 is 100% or that every future layout will pass without flags. And no domain-trained neural NER model exists yet (see Sections 14 and 16 for why).

6. Literature Review / Background Study

SWA supplied an implementation guide at the start of the project specifying the intended pipeline architecture — PDF/OCR, preprocessing, NER, relation extraction, rules/ontology, BOQ assembly, export — along with an eight-entity, six-relation schema (MATERIAL, QUANTITY, UNIT, LOCATION, DIMENSION, STANDARD, ACTION, GRADE, and relations like HAS_QUANTITY and COMPLIES_WITH), tagged with the BIOES scheme. That guide's own benchmarks target an NER F1 of 0.88 and over 85% end-to-end line-item match, but only once the training corpus reaches 50 or more real tender PDFs and 1,000 or more human-annotated sentences. These figures aren't just guesswork on the guide's part — the client's mentor confirmed them directly during a formal requirements review meeting on 11 June 2026, adding that data volume, not model architecture, is what actually drives NER accuracy, and separately confirming that GeM's standardized catalogs make closed-vocabulary validation a sound approach for GeM-sourced tenders.

One of the more important early findings came from digging into the codebase inherited at the start of this project. The existing NER model had been trained on labels generated by a script that regex-matched academic papers and video transcripts and called the result "training data." That model scored around 99% on its own synthetic test set but only about 0.43 F1 on real tender text — a fairly textbook case of a model being evaluated on the same rules it was trained to reproduce, rather than on anything resembling the real world. Tracking this down took real effort and ended up shaping the entire data-quality approach for the rest of the project (see Section 14).

7. Methodology

The project ran on what's probably best described as a hybrid orchestration model. I directed the work and broke it into tasks; AI coding agents (several different providers across the internship — Claude-based tooling, MiniMax, DeepSeek, and Hermes) did the actual implementation, working semi-autonomously against task specifications I wrote.

In practice this meant a repeating cycle. Requirements came first — the client's brief plus the formal review meeting on 11 June 2026, written up as a locked reference document. Each piece of work was then broken down into a nine-section contract (goal, context, deliverables, steps, verification commands with expected output, acceptance criteria, constraints, dependencies, and known gotchas) before being handed to an agent. Every "done" report from an agent was then independently re-verified — actually re-running the relevant commands and looking at the real diff, not just taking the report at face value. This wasn't a box-ticking step; it caught several real integrity problems over the course of the internship, described in Section 14. Anything that failed or was incomplete went back with a tighter, more specific task, and any genuine regression was

root-caused and fixed with a test added to catch it again.

If this has to be labeled with a standard methodology name, it's closest to Agile, but with an unusually strict gate between an agent claiming something is done and it actually being accepted as done — a distinction that turned out to matter a lot, for reasons covered in Section 14.

8. Tools and Technologies Used

The pipeline is written in Python 3.12. PDF text and table extraction relies on `pdfplumber`, `PyMuPDF`, and `pdfminer`; `openpyxl` handles Excel ingestion and export; the user interface is built with `Streamlit`; and the test suite (unit, integration, regression) runs on `pytest`. Data storage is entirely file-based — JSON and manifest files — since the project's scope never needed a database.

Development happened in VS Code and various terminal-based agent CLIs, with Git/GitHub for version control. The AI agents used as the primary implementation layer, always under direct supervision and verification, included Claude-based tooling, MiniMax, and DeepSeek (via the `opencode` CLI), along with Hermes CLI. Code quality was checked with `ruff` and `mypy`. Development happened on an Apple Silicon Mac using MPS acceleration; no CUDA GPU was available in this environment.

9. System Requirements

The pipeline itself is pure Python and runs on Windows, macOS, or Linux — it does not require any particular operating system. What it needs is enough RAM to comfortably process PDFs (8GB or more recommended), and, for the optional pretrained NER assist, a GPU speeds things up but is not required — it runs fine on CPU alone. On the software side: Python 3.11–3.13 (3.14 has a known incompatibility with a CLI dependency the project uses) and Git.

Development for this internship happened on macOS with Apple Silicon (see Section 8), but that was a development-environment choice, not a requirement of the software — SWA's team can deploy and run this on whatever machines they normally use, Windows included.

10. Project Planning

Phase	Focus
Requirement Analysis	Client brief review, formal requirements meeting, entity/relation schema lock
Design	Pipeline architecture (structure-first routing, rule/dictionary extraction, BOQ assembly)

Development	Iterative agent-dispatched implementation across ingestion, extraction, validation, export, and testing layers
Testing	Unit/integration/regression test suites; independent fidelity-audit tooling built specifically to prove the client's completeness requirement
Data Foundation	127-document real tender corpus collected, deduplicated, and split into frozen train/dev/test sets; annotation-review tooling built
Ongoing	Human-reviewed training data collection (in progress) → NER model training → final evaluation

11. Implementation

Ingestion and structure-first routing. For large PDFs, a fast PyMuPDF pass scans the document outline — headings, sections, annexures — before anything else happens, and only the sections likely to contain a BOQ table get passed to the more expensive extraction stage. This mirrors how a human estimator actually navigates a large tender, and it's also what the client's own domain expert confirmed was the right approach during the review meeting. A single tender can have more than one relevant section, so the implementation supports multiple BOQ ranges per document rather than betting on one best guess.

Entity extraction. A rule-based and dictionary-lookup layer pulls out the eight schema entities — material, quantity, unit, location, dimension, standard, action, grade — from table rows and surrounding text, using pattern rules and a construction-domain gazetteer. Getting multi-column PDF layouts right needed column-aware table parsing that assembles cells by their actual position on the page, rather than reading text line by line, which would otherwise interleave content from unrelated columns.

GeM catalog validation. The client's actual GeM-submitted product list (`PUBLISH_PRODUCT.xlsx`) was ingested verbatim, with full provenance tracking, and used as a closed-vocabulary reference. When a document is identified as a GeM-portal tender, extracted materials get checked against this catalog, and anything that doesn't match is flagged as a possible extraction error rather than silently accepted — the reasoning being that GeM buyers and sellers work from the same standardized list, so a mismatch is a real signal.

BOQ assembly and fidelity auditing. Extracted rows become structured `BoqRow` objects, with unit normalization handling the many spellings that show up in real Indian tenders — "Sqm", "Sq.Mtr", and "M2" all collapse to one canonical unit, for example. A dedicated fidelity-audit module (`src/domain/fidelity.py`) compares the pipeline's output against a row count derived directly from the source document — never against the pipeline's own prior output, since that would prove nothing — and produces a per-document report of what was captured, what's missing, what's extra, and what got flagged. This is the actual mechanism behind the client's fidelity requirement (R1); it's what lets that requirement be checked rather than just asserted.

Data storage is entirely file-based: a corpus manifest in JSON with a sha256 hash per document for integrity, a frozen train/dev/test split file, gold/reference annotation files, and the generated per-document audit reports.

The user interface is a Streamlit application: upload a document, the full pipeline runs, and the resulting BOQ table displays with flagged rows visually distinct, plus a banner that correctly identifies non-BOQ documents (a compliance checklist, for instance) and reports zero rows rather than inventing false extractions.

On the security side, no real tender document is committed to a public repository, gold and reference data are hash-locked so tampering can be detected, and there's a hard rule — enforced by the tooling itself, not just a policy — that only one specific named human reviewer identity can mark training annotations as verified.

12. Testing and Validation

Testing covered four layers: unit tests for individual extraction functions, unit normalization, and table classification; integration tests running the full pipeline against real corpus documents; a locked regression suite of 48 tests confirming today's verified behavior can't silently break, including checks that a document extracts identically whether processed alone or as part of a batch; and fidelity auditing, built specifically for this project, which compares extracted output against an independent source-derived row count per document.

The most recent results from that independent auditor (`scripts/audit_fidelity_per_doc.py` - -all, re-checked directly against the repository on 2026-07-16) show **10 of 10** sacred reference documents PASS and **33 of 33** BOQ-bearing corpus documents PASS (see `results/fidelity/summary.json`). On a separate independent sample of four Excel-based enquiries, row-level content match came out at 82.5% (66 of 80 rows).

It's worth being precise about what these numbers mean side by side. Auditor PASS means every source-truth row is captured or accounted for with zero silent drops and zero unmatched extras under the locked ruler — not that every free-text entity is labeled at literature F1, and not that every future unseen layout is guaranteed without human review flags. A separate completeness harness can also report zero silently dropped rows when low-confidence extractions count as flagged; that property must not be sold as content-match accuracy.

One genuine regression came up during testing and is worth describing because it's a good example of why the full-corpus check matters. A change intended to improve extraction on one document type ended up silently degrading a different document from 100% down to 46.7% fidelity, because the newly added extraction path was being preferred over the older, reliable one whenever it returned any output at all — not just when its output was actually better. Re-running the fidelity audit across the entire reference set after this change (rather than just checking the document being worked on) caught it immediately. The fix was a proper quality-comparison check between the two extraction strategies, verified against the full set before being accepted. Section 14 covers this in more detail.

13. Results and Discussion

Fidelity engineering (client requirement R1) has real, reproducible tooling behind it. The latest independent audit puts the sacred-10 reference set at **10/10 PASS** and the broader BOQ-bearing corpus at **33/33 PASS** under the capture auditor. R1 row-capture on the current corpus is closed by that measurement; generalization to never-seen documents and neural NER accuracy remain open.

GeM catalog validation (R2) is implemented and working: the 19-product catalog was ingested verbatim from the client's own file, with provenance tracked, and it's wired into the extraction paths. Structure-first extraction (R4) is implemented and handles multi-range routing, though real-world layout edge cases still turn up. On the data foundation (R3), 127 real documents have been collected, hashed, and split, and the annotation-review tooling is ready to use — but owner-verified BIOES gold data for retraining currently sits at zero.

Model accuracy is the area with the most honest ground still to cover. The primary entity-extraction layer is rule-based, optionally assisted by a generic pretrained NER model that hasn't been fine-tuned on this domain. There's no domain-trained neural model yet — that's a deliberate interim state while real labeled data gets collected, not an oversight. Product row-match on the four XLSX enquiries checked independently came out at 82.5%.

14. Challenges Faced

Inherited auto-generated training data. One of the earliest technical challenges was figuring out why an existing NER model scored around 99% on its own test data but performed poorly, around 0.43 F1, on real tenders. Investigation traced this back to a training-data generation script that had regex-matched academic papers and video transcripts and called the result "human-annotated training data." In effect, the model was being tested on the same rules it was trained to reproduce, on text that didn't resemble a real tender at all — a circular setup that no amount of further tuning could fix. The actual fix was redesigning the data pipeline to require independently sourced, genuinely human-verified labels going forward.

Verifying AI-agent work instead of trusting it. Because so much of the implementation ran through AI coding agents, a recurring and genuinely serious problem was that "task complete" reports from agents turned out, on multiple occasions, to be false or misleading. This included agents editing test expectations to match broken output rather than fixing the actual bug, and agents modifying reference or gold data to match a model's output rather than keeping it as an independent check. The only way to deal with this was to build a real verification discipline — every claimed result got independently re-derived by re-running the actual command and inspecting the actual diff before it was accepted — and to lock a set of checksum-verified "frozen" reference files specifically so tampering with the measurement system itself would be detectable.

A real extraction regression, caught before it shipped. While iterating on the pipeline, a change meant to improve one document's extraction quietly broke a different document that had previously been extracting perfectly, dropping its fidelity from 100% to 46.7%. The cause was that a newer, more sophisticated

extraction path was being preferred over the older reliable one whenever it produced any output, rather than only when its output was actually better. This surfaced because the fidelity audit was re-run across the entire reference set after every change, not just spot-checked on the document being worked on. Once root-caused, the fix was a proper quality-comparison check between the two extraction strategies, verified against the full set before being accepted.

Data availability and confidentiality. The client's real tender documents needed to be collected, organized, and split correctly — with a frozen test set that never touches training, to keep the final evaluation honest — while also being handled carefully given how client-specific and confidential they are. At one point the project's repository was briefly configured as publicly visible on GitHub; this was caught and corrected quickly.

Time management against a hard external deadline. The genuine bottleneck to full completion is the human annotation review, and that work can't be rushed or automated without undermining the whole point of having real, human-verified labels. The way through this was to be upfront about the split: the engineering and fidelity infrastructure could honestly be delivered on schedule, while the trained model itself structurally needs more review time than the internship allowed.

15. Learning Outcomes

This internship built a genuinely wide set of skills. On the technical side: systematic debugging, meaning tracing regressions and data-quality issues back to their actual root cause instead of patching symptoms; designing for data integrity and provenance, using checksums and frozen reference sets to make tampering or self-comparison detectable rather than just discouraged by policy; and test-driven, regression-driven engineering, including building a permanent suite specifically to stop previously-fixed bugs from quietly coming back.

Working with AI coding agents taught a different kind of skill — decomposing work into precisely specified, independently verifiable contracts, and holding a hard line between an agent claiming something is done and that thing actually being verified as done. Managing a multi-branch, multi-agent git history, including safely untangling a contaminated branch, was its own kind of lesson in version control at scale.

There was real domain learning too: Indian construction tender conventions, BOQ structure, the many ways units get written in practice, and how GeM's standardized catalog model works. And underlying all of it was something less technical but just as important — communicating status honestly under real deadline pressure, and being willing to say plainly what was done, what wasn't, and why, rather than rounding up.

16. Future Scope

The most important next step is human review of the queued annotation candidates. The tooling for this is already built; what remains is the actual review work — roughly 1,000 verified sentences is the target, and 32,933 candidates from the 65-document training pool are already drafted and waiting. Once that data exists, the next step is training and integrating an actual domain-specific NER model — the architecture for this

already exists in the codebase, it just needs real data to train on, which would move the project past its current rule-based extraction layer.

On the fidelity side, the immediate next step is keeping the **33/33** auditor bar green under regression as new tenders arrive, and extending the same discipline across the full 127-document corpus and future intake. Beyond that, a controlled, one-time evaluation against a frozen held-out test set would give a real NER/content-match final number, reported exactly as measured rather than adjusted after the fact. The intake pipeline is already built so that every future tender SWA receives gets the same fidelity-audit treatment automatically, with no extra setup needed.

17. Conclusion

This internship delivered a working, fidelity-instrumented pipeline that converts real construction tenders into structured BOQs, built around the client's actual acceptance requirements: independent audit tooling, GeM catalog validation, and structure-first handling of large documents. As of the 2026-07-16 full-corpus re-check, the sacred-10 reference set and the broader BOQ-bearing corpus both pass **100% of documents** under the independent row-capture auditor (**10/10** and **33/33**). A separate completeness harness can show zero silently dropped rows when low-confidence rows are counted as flagged — a real property, but a different claim from neural NER F1 or free-text content-match accuracy, and the report has tried throughout to keep those things distinct rather than blur them together for a better-sounding number.

Just as important as the numbers, though, was the discipline behind them. Verifying every claimed result independently, rather than trusting self-reported completion, caught multiple real integrity problems during development that would otherwise have gone unnoticed. The project's largest remaining piece — an actual trained NER model — is reported honestly as not yet done, gated on human-reviewed data exactly as both the source literature and the client's own guidance said it would need to be. The tooling to get there is built and ready to use.

Overall, this internship provided hands-on experience in NLP pipeline engineering, diagnosing data-quality problems, directing AI-agent-driven development responsibly, and — probably the most durable lesson of all — reporting technical status honestly even when the pressure is on to round up.

18. References

Client-provided materials:

1. SWA Consultancy Pvt. Ltd., "RFQ to BOQ Scope Extraction using NLP System" — internal project brief and implementation guide.
2. SWA Consultancy Pvt. Ltd., Requirements Review Meeting minutes, 11 June 2026.

Websites / Documentation:

- pdfplumber documentation — <https://github.com/jsvine/pdfplumber>
- PyMuPDF (fitz) documentation
- Streamlit documentation — <https://docs.streamlit.io>

19. Appendices

Source Code: available in the project's GitHub repository (private).

Weekly Progress Report

Week	Activities Completed	Skills Learned
Early phase	Client brief review, requirements-gathering meeting, entity/relation schema design	Requirement analysis, domain research
Mid phase	Pipeline architecture, extraction rules, GeM catalog integration, structure-first routing	NLP pipeline design, document parsing
Data phase	Real corpus collection (127 documents), train/test split design, annotation tooling	Data governance, provenance design
Verification phase	Fidelity-audit tooling, regression suite, integrity discipline, regression bug catch-and-fix	Testing, systematic debugging, agent-oversight discipline
Final phase	Repository cleanup, honest status reporting, final report preparation	Technical writing, professional communication

Annexure

- GitHub Repository Link — <https://github.com/Srujan0798/rfq2boq>

Internship Offer & Acceptance Letter



SWA CONSULTANCY PRIVATE LIMITED

+91 99989 44478 | viraj@swaconsultancy.com

Ground Floor, Showroom No.3, Brooklyn Tower, Next to YMCA Club,
S.G. Highway, Ahmedabad, Gujarat, India - 380051

CIN : U74900GJ2025PTC163515 | GST : 24ABQCS3694A1Z6 | PAN : ABQCS3694A

Date: 8th May 2026

Summer Internship Offer

Dear Srujan Sai,

We are pleased to extend you an offer for the position of "Project Intern" in our Company and look forward to having you as part of our team. You will be working on the project titled: "RFQ to BOQ Scope Extraction using NLP". The internship would commence from your joining date. This offer is valid till 11 May 2026.

During the term of your internship, you shall devote your full time, skill and attention to your duties and responsibilities and shall discharge your duties efficiently, diligently, and in the interest of the Company. You shall comply with all company policies and regulations.

Stipend: INR 15000 per month.

Location: Ahmedabad

Duration: 8-12 weeks

For Swa Consultancy Private Limited

SHAH VIRAJ
MITUL
Digitally signed by SHAH
VIRAJ MITUL
Date: 2026.05.08 16:54:38
+0530'

I accept the above Offer:

Start date will be: 14 May 2026

Authorized Signatory

Name: Srujan Sai

Signature: ch. Srujan Sai